

Project: Cardholder Management System (C++ Edition)

Project Overview

You will build a software system to process monthly credit card purchases. You will utilize **Inheritance, Polymorphism, File I/O, and Exception Handling**.

- **Part 1:** Build the Class Hierarchy.
 - **Part 2:** Read data from files and generate reports.
 - **Part 3:** Implement exception handling for invalid data.
-

Part 1: The Class Hierarchy

Goal: Implement an abstract base class and three derived classes to model different tiers of credit card holders.

1.1 The Base Class: `Cardholder`

Create an abstract class named `Cardholder`.

Member Variables (Protected):

- `category` (string)
- `acctNumber` (string)
- `name` (string)
- `prevBalance` (double)
- `payment` (double)
- `purchases` (`std::vector<double>`) - Note: The Java requirement forced Arrays, but `std::vector` is preferred in C++ for dynamic purchase lists.

Constants:

- `INTEREST_RATE`: static constant set to 0.01.

Methods:

- **Constructor:** Accepts Account Number and Name. Initialize `purchases` as empty.
- **Getters/Setters:** For all fields.
- `addPurchase(double price)`: Adds a single purchase to the vector.
- `addPurchases(std::vector<double> prices)`: Adds multiple purchases.

- **deletePurchases (std::vector<double> prices):** Removes found values from the list.
- **interest():** Returns $(\text{prevBalance} - \text{payment}) * \text{INTEREST_RATE}$.
- **totalPurchases():** Returns the sum of all values in the purchases vector.
- **balance():** Returns $\text{prevBalance} + \text{interest} + \text{totalPurchases}$.
- **currentBalance():** Returns $\text{prevBalance} - \text{payment} + \text{interest} + \text{totalPurchases}$.
- **minPayment():** Returns 3% of the current balance.
- **toString():** Returns a formatted string (Use `std::stringstream` to format currency).
- **purchasePoints(): Pure Virtual Function.** Returns an `int`.

C++ Hint: Virtual Destructors Since this class is intended to be a base class, you **must** declare the destructor as `virtual ~Cardholder() {}`. If you don't, deleting a derived object via a base pointer will cause undefined behavior.

1.2 Derived Class: SapphireCardholder

Inherits from `Cardholder`.

- **Constructor:** Calls base constructor; sets category to "Sapphire Cardholder".
- **purchasePoints():** Implementation: 1 point per whole dollar of total purchases.

1.3 Derived Class: DiamondCardholder

Inherits from `Cardholder`.

- **Additional Variable:** `discountRate` (double), default 0.05 (5%).
- **Constructor:** Calls base constructor; sets category to "Diamond Cardholder".
- **purchasePoints():** Implementation: 3 points per whole dollar of total purchases.
- **totalPurchases(): Override.** Apply the `discountRate` to the total.
- **toString():** Override to include the discount rate information.

1.4 Derived Class: BlueDiamondCardholder

Inherits from `DiamondCardholder` (Grandchild of `Cardholder`).

- **Additional Variable:** `bonusPoints` (int), default 2500.
- **Constructor:** Sets category to "Blue Diamond Cardholder"; sets `discountRate` to 0.10 (10%).
- **purchasePoints():** Implementation: 5 points per dollar. **Logic:** If `totalPurchases > 2500.00`, add `bonusPoints` to the total points.

Part 2: File Processing & Reporting

Goal: Read a data file containing mixed cardholder types, instantiate the correct objects, store them, and print a report.

2.1 The Data File (`card_data.txt`)

Create a text file with the following format. Each line represents one cardholder. The first integer indicates the type (1=Sapphire, 2=Diamond, 3=BlueDiamond).

Example Data:

Plaintext

```
1 10001 Smith 1200.00 200.00 34.50 100.00 63.50
2 10002 Jones 1200.00 0.00 34.50 100.00 63.50 300.00
3 10003 King 1200.00 0.00 34.50 100.00 63.50 300.00 100.00
```

2.2 The `CardholderProcessor` Class

Create a class to manage the list.

Member Variables:

- `std::vector<Cardholder*> cardholders`: A vector of **pointers** to the base class. This is required for polymorphism.

Methods:

- `readData(std::string filename):`
 1. Open the file using `std::ifstream`.
 2. Read the Type Code (int).
 3. Based on the code, dynamically allocate memory (e.g., `new SapphireCardholder(...)`).
 4. Read the common fields (Acct, Name, PrevBal, Payment).
 5. Read the remaining numbers on the line as **Purchases**.
 6. Push the pointer into the `cardholders` vector.
- `generateReport():`
 1. Iterate through the vector.
 2. Call `toString()` on each pointer (Dynamic Binding will ensure the correct child method is called).
 3. Print the result to the console.

C++ Hint: Reading Variable Length Purchases The input file has a variable number of purchases at the end of the line.

```
C++
// Logic concept
string line;
```

```

while (getline(file, line)) {
    stringstream ss(line);
    ss >> type >> acct >> name >> prevBal >> payment;
    // Create object based on type...

    double purchase;
    while (ss >> purchase) {
        currentObject->addPurchase(purchase);
    }
}

```

C++ Hint: Memory Management Because you used `new` to create objects, your `CardholderProcessor` destructor should loop through the vector and `delete` every pointer to prevent memory leaks.

Part 3: Exception Handling

Goal: Make the file reader robust. It should not crash if the file is missing or contains bad data.

3.1 Custom Exception

Create a class `InvalidCategoryException` that inherits from `std::exception` (or just a standalone class).

3.2 Error Scenarios to Handle

Modify your `readData` method to handle the following:

1. **File Not Found:** Wrap the file opening logic in a check. If `!file.is_open()`, throw a standard error or exit gracefully.
2. **Invalid Category Code:**
 - If the first integer on a line is not 1, 2, or 3, throw your `InvalidCategoryException`.
 - **Catch** this exception inside the reading loop.
 - **Action:** Print an error message (e.g., "Skipping invalid record: " + line data) and continue to the next line. Do **not** stop the program.
3. **Invalid Numeric Data:**
 - If the program expects a double (balance) but finds text (e.g., "Twelve"), `stringstream` will fail.
 - Check for parsing failures, print a warning, and skip that record.

3.3 Sorting (Optional Challenge)

Add a method to sort the `cardholders` vector before printing.

- Implement `std::sort` using a custom lambda comparator.
 - Sort by `currentBalance` descending.
-

C++ Implementation Cheat Sheet

Formatting Currency (`toString`)

Java has `DecimalFormat`. In C++, use `<iomanip>`:

```
C++
#include <sstream>
#include <iomanip>

std::string toString() {
    std::stringstream ss;
    ss << "Acct: " << acctNumber << "\n";
    ss << "Balance: $" << std::fixed << std::setprecision(2) <<
currentBalance();
    return ss.str();
}
```

Polymorphism in Action (Main Method)

```
C++
int main() {
    std::vector<Cardholder*> list;

    // Part 1 & 2 Concept
    list.push_back(new SapphireCardholder("1001", "Sam"));
    list.push_back(new DiamondCardholder("1002", "Pat"));

    // Part 2: Reporting
    for (Cardholder* c : list) {
        std::cout << c->toString() << std::endl;
        std::cout << "Points: " << c->purchasePoints() << std::endl; // Calls
correct derived logic
    }

    // Cleanup
    for (Cardholder* c : list) {
        delete c;
    }
    return 0;
}
```